Qwen

Симуляция Живого Мира: Архитектура Ticking FSM для Каннибализма NPC в Raylib

Архитектура Логики Поведения: Конечный Автомат как Фундамент

Для реализации сложной, но предсказуемой и управляемой логики поведения NPC, сфокусированной на удовлетворении базовой потребности в еде, наиболее подходящей архитектурой является конечный автомат [15]. Этот паттерн позволяет структурировать поведение персонажа как набор дискретных состояний и переходов между ними, что идеально соответствует иерархии действий, описанной в исследовательской задаче. Конечный автомат представляет собой математическую модель вычисления, которая может находиться в одном из конечного числа состояний и переходит из одного состояния в другое в ответ на входные данные [14] [15]. В контексте игровых агентов, состояниями являются действия или режимы поведения (например, "Голоден", "Ищет работу", "Покупает еду"), а переходы — условия, которые должны быть выполнены для смены действия (например, "если уровень голода > 90% и у персонажа есть деньги > цена_еды"). Такой подход обеспечивает четкую структуру и предсказуемость, что особенно важно при симуляции большого количества агентов, где необходимо избежать хаотичного и нелогичного поведения [15].

Прямая реализация простого конечного автомата с помощью `enum` и `switch`-оператора в C# возможна для небольшого количества состояний [14]. Однако, учитывая предложенную многоступенчатую цепочку действий (покупка, работа, кража, нападение), количество состояний и, соответственно, переходов быстро возрастет, что приведет к усложнению и снижению читаемости кода [15]. Для решения этой проблемы рекомендуется использовать более продвинутую версию — иерархический конечный автомат (HFSM). HFSM позволяет группировать связанные состояния в "композитные" состояния, создавая древовидную структуру [15]. Например, состояние "Поиск_ресурсов" может быть

композитным состоянием, содержащим подсостояния "Поиск_работы", "Поиск_укрываемого_места" и "Поиск_цели". Это значительно упрощает диаграмму состояний и делает систему более модульной и легкой для поддержки. Эффективная реализация HFSM в C# часто основана на наследовании: создаётся абстрактный базовый класс `BaseState`, определяющий общие методы (`Enter`, `UpdateLogic`, `Exit`), а конкретные состояния (`IdleState`, `AttackState`) наследуются от него и переопределяют необходимые методы [15]. Управление всеми этими объектами осуществляется центральным контроллером, который хранит ссылку на текущее состояние и вызывает его метод обновления каждый тик [14] [15].

Сравнение различных подходов к проектированию ИИ показывает, что для данной задачи конечные автоматы являются оптимальным выбором. Хотя машинное обучение (МО) способно создавать высоконастраиваемых и адаптивных персонажей, его применение для тысячи одновременных агентов сопряжено с огромными вычислительными затратами как на этапе обучения, так и на этапе вывода модели, что делает его малопригодным для данной задачи [15]. Конечные автоматы, напротив, обеспечивают достаточный уровень сложности для моделирования желаемого поведения при минимальных ресурсах. Они позволяют точно задать правила принятия решений, что соответствует требованию логической целостности мира, аналогично тому, как это реализовано в таких играх, как Dwarf Fortress [8] [35]. Кроме того, существуют и более декларативные подходы, такие как программирование с ограничениями, которое успешно применялось для решения сложных задач, подобных инвентаризации в Minecraft [1]. Хотя для данной задачи это может быть избыточным, сам принцип описания правил через набор ограничений, а не жестко закодированных условий, может быть полезен для создания гибкой системы, где правила могут изменяться без изменения кода. Однако для начала разработки иерархический конечный автомат предоставляет надежную и проверенную временем основу.

Подход к ИИ	Преимущества	Недостатки	Соответствие задаче
Конечный автомат (FSM)	Простота реализации, предсказуемость поведения, низкие вычислительные затраты, хорошая масштабируемость для большого числа агентов [14] [15].	Может стать сложным для поддержки при большом количестве состояний/переходов; поведение может быть слишком предсказуемым [15].	Высокое. Идеально подходит для структурирования иерархии действий и обеспечения предсказуемой логики.
Иерархический конечный автомат (HFSM)	Группировка связанных состояний, улучшение модульности и читаемости кода, возможность создания сложных, но управляемых систем [15].	Более сложная в реализации по сравнению с простым FSM, требует продуманного проектирования иерархии.	Оптимальное. Наилучшим образом подходит для моделирования многоступенчатой цепочки действий NPC.
Машинное обучение (ML)	Высокая адаптивность и способность к эмерджентному поведению [15].	Очень высокие вычислительные затраты, сложность в обучении и отладке, риск нестабильного поведения [15].	Низкое. Непрактично для масштабирования на тысячи агентов в реальном времени.
Программирование с ограничениями	Декларативный подход, гибкость в определении правил, эффективность в решении сложных комбинаторных задач [1].	Высокий порог входа, может быть избыточным для данной задачи, требует специализированных библиотек.	Условное. Может быть рассмотрено как расширение в будущем, но не для первоначальной реализации.

Таким образом, выбор в пользу иерархического конечного автомата является стратегически верным решением, которое позволит создать сложную, логически последовательную и, что самое важное, масштабируемую систему поведения для неигровых персонажей, полностью соответствующую поставленной исследовательской цели.

Реализация Тиковой Системы для Масштабирования до Тысяч Агентов

Требование обеспечить логическое существование мира для тысяч NPC одновременно является ключевым техническим вызовом проекта. Прямой и самый очевидный подход, при котором каждая сущность обновляется в каждом игровом цикле, практически гарантированно приведет к катастрофическому падению производительности [17]. Аналогичный опыт был получен в проекте Unity 6, где симуляция 60 NPC, действующих одновременно, вызывала серьезные проблемы с производительностью из-за синхронного обновления [17]. Чтобы избежать этого, необходимо внедрить систему, которая распределяет нагрузку на процессор равномерно во времени. Центральным элементом такой системы является использование тиков, но не синхронных, а рассредоточенных.

Рассредоточенная тиковая система — это архитектурный паттерн, при котором обновление каждого NPC происходит не в один и тот же момент времени, а в разные моменты, сгенерированные случайным или детерминированным образом. Вместо того чтобы все NPC были обновлены в игровом тике N, их обновления распределены по тикам N, N+k, N+2k и так далее, где k — это шаг или сдвиг для каждого конкретного NPC. Этот механизм можно легко реализовать, назначив каждому NPC уникальный идентификатор и используя его для расчета следующего времени обновления. Например, `nextUpdateTick = currentTick + baseInterval + (npcId % randomJitter)`. Такой подход эффективно "размазывает" нагрузку по всему игровому процессу, предотвращая пиковые значения потребления CPU и обеспечивая стабильную частоту кадров [17]. Эта техника является фундаментальной для любой игры, стремящейся к симуляции больших масс агентов, и была бы применима даже в рамках стандартных игровых движков, таких как Godot, где можно управлять частотой обновления физики и логики независимо [62].

Для организации и управления этим процессом можно выделить два ключевых компонента: глобальный диспетчер агентов и внутреннюю систему тиков каждого NPC. Глобальный диспетчер будет отвечать за запуск основного игрового цикла и периодическую проверку, какие именно NPC готовы к обновлению. Он может хранить очередь или список всех активных NPC, отсортированных по их следующему времени обновления. В каждом тике диспетчер проверяет, нет ли в списке NPC, чье время обновления совпало с текущим. Если таковые имеются, он передает управление их внутренней системе тиков. Внутренняя система каждого NPC, в свою очередь, содержит всю необходимую логику поведения, реализованную через иерархический конечный автомат (HFSM), описанный ранее. Когда диспетчер активирует NPC, его HFSM получает команду на обновление. HFSM проверяет условия для перехода между состояниями (например, "стал ли я достаточно голодным, чтобы начать искать еду?") и выполняет действия, соответствующие текущему состоянию (например, "передвигайся к магазину").

Эта двухуровневая архитектура позволяет достичь нескольких важных преимуществ. Во-первых, она обеспечивает максимальную масштабируемость. Добавление нового NPC в мир добавляет лишь незначительную дополнительную нагрузку на диспетчера, но не увеличивает пиковую нагрузку на CPU в каждый тик. Во-вторых, она обеспечивает глобальную логическую автономность. Поскольку каждый NPC обновляется по своему собственному графику, они все равно будут принимать решения и взаимодействовать с

миром, даже если игрок находится далеко от них. Их логика продолжает работать, просто с меньшей частотой обновления, чем у NPC в зоне видимости камеры (которые, возможно, будут обновляться чаще для более плавного поведения). В-третьих, эта система хорошо сочетается с другими оптимизациями, такими как управление жизненным циклом объектов и сборка мусора. Поскольку обновление происходит дифференцированно, меньше объектов активно используются и создаются в один и тот же момент времени, что помогает снизить давление на сборщик мусора . Таким образом, рассредоточенная тиковая система является не просто опцией, а обязательным компонентом архитектуры для создания живого и масштабируемого мира, о котором мечтал пользователь.

Проектирование Модели NPC и Состояний Поведения

Для успешной реализации цикла поведения на основе иерархического конечного автомата (HFSM) требуется тщательное проектирование данных NPC и четкое определение его состояний. Модель данных должна быть легковесной, эффективной в памяти и содержать всю информацию, необходимую для принятия решений. Состояния, в свою очередь, должны точно отражать иерархию действий, описанную в запросе, и обеспечивать плавные переходы между ними.

Модель данных NPC (или его компоненты в парадигме ECS) должна включать следующие ключевые поля:

- **ID:** Уникальный идентификатор для каждого NPC, необходимый для рассредоточенной тиковой системы и для идентификации в мире [14].
- **Position:** Координаты в игровом мире, необходимые для поиска путей и взаимодействия с объектами.
- **Stats (**Статистика**):** Коллекция основных характеристик, влияющих на поведение. Это могут быть физические показатели, такие как "сила", "выносливость", а также интеллектуальные, как "сообразительность". Эти параметры будут влиять на шансы успеха при нападении или эффективность на рабочем месте [25].
- **Skills (**Навыки/Компетенции**):** Список или набор компетенций, таких как "Воровство", "Сила", "Шахтерство", "Садоводство". Эти навыки будут

сравниваться с требованиями к вакансиям или уровнем защиты объектов. Можно представить их как числовые значения или как набор булевых флагов [10].
- **Needs (Потребности):** Словарь или структура, описывающая текущее состояние базовых потребностей. В данном случае главной потребностью будет "Голод", представленная значением от 0 до 100, где 100 означает максимальный голод. Также здесь могут храниться другие, пока неиспользуемые, потребности (например, "Усталость", "Жажда").
- **Inventory (Инвентарь):** Структура, содержащая предметы, которые NPC владеет. Важнейшими предметами будут "Еда" и "Деньги". Проектирование инвентаря само по себе может быть сложной задачей, и опыт из RimWorld, где используется C# и сложные механики, может служить примером [11] [18].
- **Money (Деньги):** Числовое значение, представляющее количество наличных денег у NPC.
- **CurrentState (Текущее состояние):** Ссылка на объект, представляющий текущее состояние HFSM (например, `IState attackState = new AttackState()`). Это позволяет легко менять поведение NPC, просто присваивая этому полю другой объект состояния [15].
- **Target (Цель):** Ссылка на целевой объект (другой NPC, производство, магазин), с которым NPC взаимодействует в данный момент.
- **Path (Путь):** Список точек, представляющий путь от текущей позиции NPC до цели. Необходим для перемещения по миру.

На основе этих данных и предложенной иерархии действий можно определить следующие состояния для HFSM:

ID Состояния	Название Состояния	Описание и Логика
S_Hungry	Голоден	Это начальное или базовое состояние. Персонаж постоянно проверяет свой уровень голода. Если он превышает некий порог, он инициирует поиск еды.
S_BuyingFood	Покупка еды	Персонаж выбирает ближайший доступный магазин. Проверяет наличие еды в наличии и свои финансовые возможности. Если оба условия выполнены, совершает покупку и переходит в состояние S_Eating. Если нет, переходит в состояние S_SearchingForWork.
S_SearchingForWork	Поиск работы	Персонаж сканирует объявления о работе в ближайших городах. Он ищет вакансии, требования к которым соответствуют его компетенциям (например, "сила" >= 10). Если подходящая работа найдена, он отправляется на нее и переходит в состояние S_Working. Если нет, переходит в состояние S_StealingFood.
S_Working	Работает	Персонаж находится на рабочем месте и работает определенное количество игровых тиков. По завершении рабочего дня он получает зарплату, возвращается домой и снова пытается купить еду (переход в S_BuyingFood).
S_StealingFood	Кража еды	Персонаж идентифицирует ближайшее производство еды (ферма, сад) и оценивает возможность кражи. Вероятность успеха зависит от его навыка "Воровство" и уровня защиты производства. При успехе он забирает часть еды и переходит в состояние S_Eating. При неудаче он может получить штраф или просто потерять время и снова пытаться найти ресурс.
S_AttackingNPC	Нападение на NPC	Если все предыдущие попытки провалились, персонаж переходит в это агрессивное состояние. Он выбирает другого NPC в качестве цели. Шанс победы рассчитывается на основе силы атакующего и защищающегося, а также используемого ими оружия.
S_Cannibalizing	Каннибализм	Это крайнее состояние, активируемое только после успешного нападения, если у жертвы не было еды. Персонаж начинает есть тело жертвы для удовлетворения голода.

Переходы между этими состояниями управляются логикой внутри самого HFSM-контроллера. Например, в состоянии S_BuyingFood на каждом тике вызывается метод Update(), который проверяет условия для перехода в S_Eating (еда куплена) или S_SearchingForWork (денег не хватило). Этот подход позволяет разделить логику каждого действия на независимые блоки, что упрощает отладку и дальнейшее развитие системы.

Интеграция Поиска Ресурсов и Взаимодействия с Миром

Реализация цикла поведения NPC требует, чтобы каждый из его состояний мог эффективно взаимодействовать с игровым миром для поиска и получения ресурсов. Это включает в себя не только логику принятия решений, но и

практические механизмы, такие как поиск пути, взаимодействие с объектами окружения и управление состоянием этих объектов. Ключевыми для этого являются алгоритмы поиска пути и реактивная система событий.

Поиск пути является фундаментальной технологией для любого мобильного агента в 2D- или 3D-пространстве. Для данного проекта на Raylib, который ориентирован на 2D-игры, наиболее подходящим алгоритмом является A *(A-star)*. *A* является оптимальным и полным алгоритмом поиска пути, что означает, что он всегда найдет кратчайший путь, если он существует, и не будет "зацикливаться" бесконечно. Его эффективность достигается за счет использования эвристической функции, которая оценивает стоимость прохождения от текущей точки до цели, что позволяет алгоритму делать осмысленный выбор направления. В контексте нашего проекта, поиск пути будет активироваться в нескольких состояниях HFSM: `S_BuyingFood` (движение к магазину), `S_SearchingForWork` (движение к городу с вакансиями), `S_StealingFood` (движение к производству) и `S_AttackingNPC` (движение к цели). Реализация A* в C# является стандартной задачей и широко документирована. Получив путь, NPC будет последовательно перемещаться по точкам этого пути до достижения своей цели.

Помимо движения, NPC должен уметь взаимодействовать с объектами в мире. Это взаимодействие должно быть основано на событиях, а не на постоянном опросе. Например, вместо того чтобы NPC в состоянии `S_BuyingFood` постоянно проверял, есть ли в магазине еда, магазин сам должен генерировать событие "Товар_куплен" или "Товар_закончился". NPC, подписанные на эти события, смогут отреагировать на них. Эта концепция, известная как система событий или сигналов, является мощным инструментом для создания слабосвязанной и масштабируемой архитектуры [60]. Она позволяет отделить логику NPC от логики мира. Когда NPC решает купить еду, он отправляет запрос в магазин. Магазин обрабатывает запрос и, в зависимости от результата, генерирует соответствующее событие. NPC, которому этот запрос был сделан, получает результат и решает, какой шаг предпринять дальше.

Эта реактивная модель также применима к другим взаимодействиям. Когда NPC успешно крадет еду, производство (ферма) получает событие "Еда_украдена" и обновляет свое состояние (уменьшает количество ресурсов). Когда NPC нападает на другого NPC, последний получает событие "Получен_урон" и может ответить событием "Нападение_отбито" или "Убит". Такой подход делает мир более живым и реалистичным. Изменения в состоянии одного объекта немедленно распространяются на весь мир, и другие

агенты могут на них реагировать. Это также предотвращает ситуацию, когда множество NPC одновременно пытаются купить последнюю единицу еды в магазине, что может привести к состоянию гонки. Событийная модель гарантирует, что операции будут обрабатываться последовательно.

Наконец, необходимо предусмотреть механизмы для оценки шансов успеха в различных действиях. Эти оценки должны быть основаны на статистике и навыках NPC, а также на характеристиках цели или объекта. Например:

- Успешность покупки: $Success_{buy} = (Money \geq Price_{food}) \wedge (Stock_{food} > 0)$.
- Успешность кражи: $Success_{steal} = Random(0,1) < \frac{Skill_{thievery}}{Difficulty_{location}}$. Здесь `Random(0, 1)` генерирует случайное число от 0 до 1, имитируя шанс [51]. `Difficulty` может зависеть от типа производства (ферма имеет низкий уровень защиты, торговая галерея — высокий).
- Успешность нападения: $Win_{fight} = Random(0,1) < \frac{Strength_{attacker} \times Weapon_{attacker}}{Strength_{defender} \times Weapon_{defender}}$. Эта формула позволяет моделировать бои, где более сильный или вооруженный NPC имеет преимущество.

Эти формулы могут быть значительно усложнены в будущем, но уже на этом этапе они создают основу для динамичного и непредсказуемого, но все же логичного поведения NPC. Реализация этих механизмов через событийную модель и алгоритмы поиска пути позволит создать глубокую и увлекательную симуляцию, где мир действительно живет своей жизнью.

Оптимизация Производительности: От Памяти до Сборки Мусора

Для обеспечения работы тысячи NPC без ущерба для производительности, оптимизация должна стать неотъемлемой частью всего процесса разработки, а не просто заключительным этапом. Основные узкие места в такой системе — это вычислительная нагрузка на ЦП и использование памяти, в особенности давление на сборщик мусора в среде .NET. Стратегия оптимизации должна быть комплексной и охватывать как архитектуру, так и конкретные практики написания кода.

Первым и самым важным шагом является минимизация аллокаций памяти в основном игровом цикле. Каждый раз, когда в C# создается новый объект типа `class` (например, `new Vector2()` или `new string()`), выделяется память на куче (managed heap). Частое создание и уничтожение таких объектов приводит к постоянной работе сборщика мусора, который периодически "замораживает" выполнение программы, чтобы очистить неиспользуемую память. Эти паузы, хотя и короткие, могут заметно сказаться на плавности игры, особенно при тысячах одновременных агентов. Ключевым решением этой проблемы является использование структур вместо классов там, где это возможно [90]. Структуры — это типы значений, которые по умолчанию выделяются на стеке, а не на куче, и их создание/уничтожение гораздо дешевле. Например, статистику NPC (`Stats`) или его положение (`Position`) следует представлять не как объекты класса, а как структуры. Использование `struct` для легковесных данных, составляющих сущность, является одной из лучших практик для снижения давления на GC [90].

Вторым мощным инструментом является использование `Span<T>` и `Memory<T>`. Эти типы, представленные в современных версиях .NET, позволяют работать с непрерывными регионами памяти (будь то массивы, фрагменты массивов или участки стека) без необходимости создавать копии данных или выделять новую память [87] [88]. Например, при обработке данные о состоянии NPC или при работе с данными карты можно использовать `Span<float>`, чтобы передавать и манипулировать ими эффективно, избегая выделения памяти. Это особенно актуально для производительностных критичных операций.

Третий аспект — это использование пулов объектов, таких как `ArrayPool<T>`. Хотя структуры решают проблему создания множества мелких объектов, иногда возникает необходимость создавать большие массивы или другие крупные объекты. Вместо того чтобы создавать и уничтожать такие объекты многократно, можно взять экземпляр из пула, использовать его и вернуть обратно, когда он больше не нужен. Это позволяет повторно использовать память, избегая дорогостоящих операций выделения на куче [90]. Для многих стандартных типов, таких как массивы, .NET предоставляет встроенные пулы, которые можно эффективно использовать.

Четвертый, и самый важный с практической точки зрения, шаг — это профилирование. Теоретические знания об оптимизации бесполезны, если они направлены на неверные участки кода. Необходимо использовать инструменты профилирования, чтобы "измерять, а не догадываться" [48]. Как показано в опыте с проектом Unity 6, первоначальная догадка о причине проблемы может

оказаться неверной [17]. Только после анализа данных профилировщика стало понятно, что проблема заключалась не в самих NPC, а в том, что все они были обновлены в один и тот же момент времени. Профилирование позволяет выявить самые медленные функции, определить, какие операции занимают больше всего времени, и где происходят наибольшие объемы аллокаций. Это дает четкое указание на то, на чем стоит сфокусировать усилия по оптимизации [17] [48].

Наконец, нельзя забывать и о самом движке Raylib. Хотя он и является легковесным, его функции рендеринга и обработки ввода также потребляют ресурсы. Хотя основная задача пользователя — симуляция мира, а не рендеринг, важно помнить, что вся система должна работать в рамках одного игрового цикла. Использование отдельных таймеров для логики и рендеринга, как это возможно в некоторых движках [68], может быть полезным, но в рамках Raylib основной фокус должен быть на оптимизации самой логики симуляции, поскольку именно она является самым ресурсоемким компонентом при работе с тысячами агентов.

Техника оптимизации	Описание	Преимущества	Недостатки
Использование **struct**	Замена классов на структуры для легковесных данных.	Снижение давления на сборщик мусора (GC), так как данные выделяются на стеке [90].	Ограничения по размеру (более 16 байт), поведение при передаче по значению [90].
Использование **Span<T>** / **ArrayPool<T>**	Работа с областями памяти без выделения новых объектов.	Нулевая аллокация при манипуляции данными, повторное использование больших массивов [87] [90].	Более сложная логика по сравнению с обычными коллекциями.
Рассредоточенная тиковая система	Обновление NPC в разное время для распределения нагрузки на ЦП.	Предотвращение пикового потребления ЦП, обеспечение стабильной производительности [17].	Может увеличить время реакции на некоторые события.
Профессиональное профилирование	Использование специализированных инструментов для анализа производительности.	Точные данные о "узких местах", возможность сфокусировать усилия на реальных проблемах [17] [48].	Требует времени и навыков для освоения инструментов.

Применение этих техник в комплексе позволит создать систему, которая не только будет выполнять все требуемые функции, но и будет работать эффективно и плавно даже при очень большой плотности населения.

Синтез и Комплексная Стратегия Реализации

На основе всестороннего анализа предоставленных материалов и требований исследования, формируется комплексная стратегия разработки цикла поведения NPC. Данная стратегия объединяет архитектурные решения, методы оптимизации и логические построения в единую, последовательную дорожную карту для реализации проекта. Цель состоит в создании автономного, масштабируемого и логически целостного симулятора, где каждый NPC самостоятельно заботится о своей базовой потребности в еде, независимо от присутствия игрока.

Шаг **1:** Архитектурное Ядро — Иерархический Конечный Автомат **(HFSM)** Основой системы поведения станет иерархический конечный автомат, реализованный в C#. Этот выбор обусловлен необходимостью структурировать сложную, многоступенчатую логику принятия решений NPC [15]. Каждое состояние в этой системе будет представлено отдельным классом, унаследованным от общего интерфейса `IState` или абстрактного базового класса. Такой подход, где каждый объект состояния отвечает за свою логику, позволяет декомпозировать сложную проблему на управляемые части [14] [15]. Центральный контроллер AI будет управлять всеми NPC, храня ссылку на текущее состояние каждого из них и вызывая его метод обновления. Это обеспечит гибкость и легкость модификации поведения в будущем.

Шаг **2:** Обеспечение Масштабируемости — Рассредоточенная Тиковая Система Для поддержки тысяч NPC одновременно необходимо отказаться от синхронного обновления и внедрить рассредоточенную тиковую систему. Каждый NPC будет иметь свой собственный "таймер" обновления, сгенерированный на основе его уникального ID. Это распределит вычислительную нагрузку на ЦП равномерно во времени, предотвращая пиковые значения и обеспечивая стабильную производительность [17]. Глобальный диспетчер будет отслеживать время обновления каждого NPC и активировать только тех, чья очередь наступила. Эта система является ключевым фактором для достижения цели создания "живого" мира, который функционирует независимо от игрока.

Шаг **3:** Проектирование Модели и Логики Поведения Модель данных NPC должна быть легковесной и эффективной. Использование `struct` для хранения статистики, навыков и координат поможет минимизировать аллокации памяти на куче, что критически важно для снижения нагрузки на сборщик

мусора [90]. Логика поведения будет организована в соответствии с предложенной иерархией действий, состоящей из следующих ключевых состояний: Голоден, Покупка_еды, Поиск_работы, Работает, Кража_еды, Нападение_NPC и Каннибализм. Переходы между этими состояниями будут строго регламентированы условиями, основанными на данных NPC (уровень голода, деньги, навыки) и состоянии окружающего мира (доступность товаров, наличие вакансий).

Шаг **4:** Реактивное Взаимодействие с Миром Вместо пассивного опроса мира, система должна быть реактивной. Взаимодействие с объектами, такими как магазины, производства и другие NPC, будет осуществляться через систему событий. Когда NPC совершает действие (например, покупает еду или крадет урожай), он генерирует соответствующее событие. Объекты в мире реагируют на эти события, обновляя свое состояние. Это делает мир более динамичным и реалистичным, а также снижает количество проверок и условных операторов в основном коде AI.

Шаг **5:** Оптимизация на Каждом Этапе Оптимизация должна быть встроена в процесс разработки. Ключевыми мерами будут: постоянное профилирование для выявления "узких мест" [48]; использование `struct` и `Span<T>` для минимизации аллокаций памяти [87] [90]; и применение паттерна "пул объектов" для повторного использования ресурсов. Алгоритм поиска пути A* будет использоваться для всех мобильных действий NPC, обеспечивая эффективное перемещение по карте.

Шаг **6:** Итерационная Реализация Реализацию следует проводить итерационно. Начать следует с базовой модели NPC и одного состояния (например, Голоден). Затем постепенно добавлять новые состояния и логику переходов, постоянно тестируя производительность. После того как базовая работа NPC будет реализована, следует сосредоточиться на оптимизации всей системы в целом. Этот подход позволит контролировать сложность проекта и своевременно выявлять проблемы с производительностью.

В заключение, предложенная стратегия представляет собой комплексный и технически обоснованный план, который позволяет достичь поставленной исследовательской цели. Она сочетает в себе проверенные игровые разработкой паттерны (HFSM, тиковая система) с передовыми практиками оптимизации производительности (.NET, сборка мусора), создавая прочный

фундамент для построения сложной и автономной симуляции мира на базе Raylib и C#.

Справка

1. How I learned constraint programming with Minecraft inventory https://www.linkedin.com/posts/robert-wendt-150239186_using-constraint-satisfaction-to-optimize-activity-7383227825832513536-hp9b
2. 平台和工具 - 架构师 https://architect.pub/book/export/html/36
3. Large scale battles (at least 100000 extremely high quality 3D ... https://discussions.unity.com/t/large-scale-battles-at-least-100-000-extremely-high-quality-3d-animated-characters-with-ai/892889
4. [PDF] Declarative Specification of Temporal Entities for Large-Scale ... https://ojs.aaai.org/index.php/AIIDE/article/download/36809/38947/40886
5. GPU optimization — Godot Engine (stable) documentation in English https://docs.godotengine.org/en/stable/tutorials/performance/gpu_optimization.html
6. Stable World Models for Robotics: Interactive Simulator Released https://www.linkedin.com/posts/yixuan-wang-54298115a_robotics-ai-machinelearning-activity-7435402552856268800-oRRZ
7. Transform Your Coding with OpenAI Symphony - TikTok https://www.tiktok.com/@github.awesome/video/7613770064650636557
8. (PDF) Dwarf Fortress: Laboratory and Homestead - ResearchGate https://www.researchgate.net/publication/281832877_Dwarf_Fortress_Laboratory_and_Homestead
9. Dwarf Fortress Intermediate Guide: Strike The Earth Like a Master https://steamcommunity.com/sharedfiles/filedetails/?id=2899235556
10. Reverse Engineering and Runtime Systems Through Game Modding https://www.linkedin.com/pulse/reverse-engineering-runtime-systems-through-game-look-%C3%A7etinkaya-3kgrf
11. 天耀的1.6原版优化辅助模组合集 - Steam Community https://steamcommunity.com/sharedfiles/filedetails/?id=3519866585
12. Progression: Education - Workshop - Steam Community https://steamcommunity.com/sharedfiles/filedetails/?id=3590863552

13. Introduce Yourself! - Learn Content - Unity Discussions https://discussions.unity.com/t/introduce-yourself/816532
14. Implementing finite-state-machine AI. (C#) - Unity Discussions https://discussions.unity.com/t/implementing-finite-state-machine-ai-c/561342
15. (PDF) Finite State Machine in Game Development - ResearchGate https://www.researchgate.net/publication/355518086_Finite_State_Machine_in_Game_Development
16. Can you create a Finite State Machine using SOLID principles https://stackoverflow.com/questions/74155008/can-you-create-a-finite-state-machine-using-solid-principles
17. #unity6 #gamedev #softwarearchitecture #behaviorgraphs #csharp ... https://www.linkedin.com/posts/lucaskinoo_unity6-gamedev-softwarearchitecture-activity-7419020020917100544-L0bd
18. b站主播奶崽璇璇的超稳定超流畅合集（要求内存在16g（包括）以上） https://steamcommunity.com/sharedfiles/filedetails/?l=finnish&id=3610944989
19. Demian Brecht - Salesforce | LinkedIn https://ca.linkedin.com/in/demianbrecht
20. Inventory System Using ECS? - Unity Discussions https://discussions.unity.com/t/inventory-system-using-ecs/765016
21. Steam 创意工坊::Rimworld mods 1.6 https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3497544341
22. Steam 工作坊::TangTyrone的通用环世界合集1.13 https://steamcommunity.com/sharedfiles/filedetails/?l=tchinese&id=3645340978
23. Steam 创意工坊::Storing my old mods https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3469451868
24. Steam 创意工坊::my somewhat balanced rimworld https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3224061345
25. Steam 创意工坊::1.6 2025 https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3554981923
26. Steam Workshop::备份 https://steamcommunity.com/sharedfiles/filedetails/?id=3497710031
27. Steam 创意工坊::afsa https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2848179710
28. Steam 创意工坊::E https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2175717640
29. Debian -- Prospective packages https://www.debian.org/devel/wnpp/prospective
30. 汉堡查克- Workshop https://steamcommunity.com/sharedfiles/filedetails/?id=3639935167

31. Cektitle | PDF | Microsoft Sql Server - Scribd https://www.scribd.com/document/752533028/cektitle
32. [PDF] Approximate computing for embedded machine learning - HAL theses https://theses.hal.science/tel-03272594/file/96723_YANG_2021_archivage.pdf
33. [PDF] Huawei Research Issue 6 https://www-file.huawei.com/admin/asset/v1/pro/view/3cc66468728c4b5e8b4d48f1ee6ec3a4.pdf
34. (PDF) `New Game Physics: Added Value for Transdisciplinary ... https://www.academia.edu/5581935/New_Game_Physics_Added_Value_for_Transdisciplinary_Teams_Ph_D_Thesis
35. Game Design Foundations | PDF - Scribd https://www.scribd.com/document/405139921/167402865-Game-Design-Foundations
36. [PDF] Declarative Specification of Temporal Entities for Large-Scale ... https://ojs.aaai.org/index.php/AIIDE/article/download/36809/38947
37. Dwarf Fortress Quickstart Guide | PDF | Keyboard Shortcut - Scribd https://www.scribd.com/document/312659352/Dwarf-Fortress-Quickstart-Guide
38. Questions List - Unity Code Monkey https://unitycodemonkey.com/question_list.php
39. Unreal Engine 5.4 Release Notes - Epic Games Developers https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.4-release-notes?application_version=5.4
40. Artists Create Games with AI Coding Assistance - LinkedIn https://www.linkedin.com/posts/mightybeargames_what-happens-when-artists-become-game-developers-activity-7406389918853189632-Uk3k
41. Creating the enemy - Godot Docs https://docs.godotengine.org/en/stable/getting_started/first_2d_game/04.creating_the_enemy.html
42. (PDF) Fully Automated Photogrammetric Data Segmentation and ... https://www.researchgate.net/publication/338557943_Fully_Automated_Photogrammetric_Data_Segmentation_and_Object_Information_Extraction_Approach_for_Creating_Simulation_Terrain
43. Green Mana Inc's Tower Defense Prototype in Unity - LinkedIn https://www.linkedin.com/posts/green-mana-inc_tower-defense-prototype-activity-7421287159166586880-dNu1
44. Lord of Rigel Official Guide - Steam Community https://steamcommunity.com/sharedfiles/filedetails/?id=3291496385
45. Homebrew Core Formulae Overview | PDF | Command Line Interface https://www.scribd.com/document/551715438/Brew-Formula
46. Voraussichtliche Pakete - Debian https://www.debian.org/devel/wnpp/prospective.de.html

47. 虚幻引擎5.1版本说明 https://dev.epicgames.com/documentation/zh-cn/unreal-engine/unreal-engine-5.1-release-notes?application_version=5.1
48. CPU optimization — Godot Engine (4.5) documentation in English https://docs.godotengine.org/en/4.5/tutorials/performance/cpu_optimization.html
49. PHILIP HANSSON's Post - LinkedIn https://www.linkedin.com/posts/philip-hansson-36bb8a149_this-is-spot-on-especially-for-a-lot-of-activity-7424719954442158080-ca9T
50. ISEV2024 Abstract Book - 2024 - Journal of Extracellular Vesicles https://isevjournals.onlinelibrary.wiley.com/doi/10.1002/jev2.12444
51. Lots of Ways to Use Math.random() in JavaScript - CSS-Tricks https://css-tricks.com/lots-of-ways-to-use-math-random-in-javascript/
52. Plugins - Obsidian https://obsidian.md/plugins
53. Workshop Steam::ziyong1.6 https://steamcommunity.com/sharedfiles/filedetails/?l=french&id=3527233763
54. Sticky:EVE API and Public CREST discussion - EVE Search https://eve-search.com/thread/332097-1/page/all
55. Block'hood - Developing an Architectural Simulation Video Game https://www.researchgate.net/publication/367694869_Block'hood_-_Developing_an_Architectural_Simulation_Video_Game
56. Cultivating Pluralism In Algorithmic Monoculture - arXiv https://arxiv.org/html/2507.09650v2
57. Kibele-ESIA-7th-June-2018-Final-Report-for-Disclosure.txt https://documents1.worldbank.org/curated/en/583651531806893586/txt/Kibele-ESIA-7th-June-2018-Final-Report-for-Disclosure.txt
58. Computer Science - arXiv https://arxiv.org/list/cs/new
59. [PDF] UC Santa Cruz - eScholarship https://escholarship.org/content/qt4mr6c29j/qt4mr6c29j.pdf
60. Instancing with signals - Godot Docs https://docs.godotengine.org/en/latest/tutorials/scripting/instancing_with_signals.html
61. Mastering ROS 2 for Robotics Programming | PDF - Scribd https://www.scribd.com/document/912727269/Mastering-ROS-2-for-Robotics-Programming-Design-Build-Simulate-And-Prototype-Complex-Robots-Using-the-Robot-Operating-Lentin-Joseph-Jonathan-Cac
62. Using physics interpolation - Godot Docs https://docs.godotengine.org/en/4.4/tutorials/physics/interpolation/using_physics_interpolation.html
63. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/3-540-45449-7.pdf

64. Workshop Steam::没空间了 https://steamcommunity.com/sharedfiles/filedetails/?l=french&id=3659822450
65. perforations: Before the Rem/a/inder of the [uncanny] Event-Calls ... https://www.academia.edu/9282824/perforations_Before_the_Rem_a_inder_of_the_uncanny_Event_Calls_Comings_and_Desiderata_A_compilation_1991_2014
66. 1.6大乱炖 https://steamcommunity.com/sharedfiles/filedetails/?l=swedish&id=3518068291
67. Entertainment Computing and Serious Games - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-030-34644-7.pdf
68. Game loop with separate timer for rendering and game logic https://stackoverflow.com/questions/7920835/game-loop-with-separate-timer-for-rendering-and-game-logic
69. All Legal Scrabble Words | PDF | Leisure | Nature - Scribd https://www.scribd.com/doc/121740982/All-Legal-Scrabble-Words
70. list.txt - Columbia University Computer Science http://www.cs.columbia.edu/~tal/3261/fall07/handout/list.txt
71. 1.6大乱炖- Майстерня - Steam Community https://steamcommunity.com/sharedfiles/filedetails/?l=ukrainian&id=3518068291
72. [PDF] UNIVERSITY OF CALIFORNIA SANTA CRUZ CRAFTING STORIES ... https://escholarship.org/content/qt6nw5x48d/qt6nw5x48d_noSplash_8872dfb7173f44811537b2c243efa72b.pdf
73. Evaluating Agentic Optimization on Large Codebases - ResearchGate https://www.researchgate.net/publication/402611953_Evaluating_Agentic_Optimization_on_Large_Codebases
74. Steam Workshop::Immersive Mods - RimWorld https://steamcommunity.com/sharedfiles/filedetails/?id=2794617699
75. [PDF] A Short History of Computer Science - Dr. Yves J. Hilpisch https://hilpisch.com/history.pdf
76. Cultivating Pluralism In Algorithmic Monoculture: The Community ... https://arxiv.org/html/2507.09650v1
77. Artificial Intelligence - arXiv https://arxiv.org/list/cs.AI/new
78. [PDF] 4th National Conference on Advances in Engineering and ... - ijsrst https://ijsrst.com/paper/v5i7.pdf
79. 1.6大乱炖 https://steamcommunity.com/sharedfiles/filedetails/?id=3518068291
80. (PDF) Libro como ser un geek - Academia.edu https://www.academia.edu/109825324/Libro_como_ser_un_geek

81. HCI International 2025 Posters - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-94156-6.pdf
82. CRC.level.design.processes.and.Experiences.1138628808 https://pdfcoffee.com/crcleveldesignprocessesandexperiences1138628808-4-pdf-free.html
83. Steamワークショップ::TangTyrone的通用环世界合集1.13 https://steamcommunity.com/sharedfiles/filedetails/?l=japanese&id=3645340978
84. Digital Resources Overview | PDF | Artificial Intelligence - Scribd https://www.scribd.com/document/946229849/Copy-of-Custom-GPi
85. Interactive Storytelling - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-47655-6.pdf
86. How to handle many updating objects efficiently in C#? https://stackoverflow.com/questions/2335883/how-to-handle-many-updating-objects-efficiently-in-c
87. C# Memory Spans and Performance-Critical Code https://dev.to/chakewitz/c-memory-spans-and-performance-critical-code-219l
88. C# - All About Span: Exploring a New .NET Mainstay https://learn.microsoft.com/en-us/archive/msdn-magazine/2018/january/csharp-all-about-span-exploring-a-new-net-mainstay
89. C# Interpretation and the GC (garbage collection) counter https://stackoverflow.com/questions/10466961/c-sharp-interpretation-and-the-gc-garbage-collection-counter
90. Reducing Garbage Collector (GC) Pressure in .NET https://dev.to/adrianbailador/reducing-garbage-collector-gc-pressure-in-net-practical-patterns-and-tools-5al3